DSA4213 Assignment 1: Language Models, Embeddings and Representations

Anonymous submission

Code: https://github.com/XenonLyn/assignment-dsa4213

Comparison question. *On one small corpus and a fixed training budget, how do n-gram, RNN, LSTM and Transformer language models differ; do fixed pre-trained input embeddings help; and does better next-token prediction give better representations for classification?* Throughout, we label statements as observations (OBS) or hypotheses (HYP); every number is from a single run (seed 42) unless stated otherwise.

1 Data and common setup

Corpus. WikiText-2 [1], tokenised version (`wikitext-2-v1`); 2.08M / 216k / 244k train / validation / test tokens. Preprocessing: lower-casing (for GloVe coverage in Part II), whitespace tokens, an <eos> token at the end of every non-empty line. The vocabulary (28,913 types) is built on the training split only; the dataset's own <unk> covers 2.6% / 5.4% / 6.2% of train / validation / test tokens and is scored like any other word (no further OOV words occur). **Common evaluation.** Every split is cut into non-overlapping chunks of 128 targets; each target is predicted from the preceding tokens *inside its chunk only*, for every model including the n-gram (no state is carried across chunks). Padded targets are excluded from the loss. Metric: mean next-token cross-entropy (CE, nats) and perplexity PPL= exp(CE). All selection (n-gram order, best epoch, learning rate, classifier regularisation) uses validation data only; test data is used once per model. **Hardware and budget.** Apple M1 laptop (8-core CPU, 16 GB, fanless), PyTorch 2.5.1, neural models on the MPS GPU backend, n-gram on CPU. Every neural model gets the same budget: 10 epochs = 5,070 updates of 32×128 tokens, AdamW (weight decay 0.01), 200 warm-up steps then cosine decay to 10% of the peak learning rate (LR), gradient clipping at 1.0. Models are trained one at a time.

2 Part I: four language models

Models (details in Appendix A). *n-gram*: interpolated Kneser–Ney [2, 3] with absolute discounts per order; orders 1–4 scored on validation (PPL 818 / 275 / 236 / 230), order 4 selected. *RNN*: 2-layer tanh RNN, hidden 512. *LSTM* [4]: 2 layers, hidden 512. *Transformer* [5]: decoder-only, 4 pre-LN blocks, d=300, 6 heads, FFN 1200, learned positions, causal mask. All neural models use randomly initialised, trainable 300-d input embeddings (chosen to match GloVe-300d for Part II), an untied softmax layer, and dropout (0.3 RNN/LSTM, 0.2 Transformer) on the embeddings, between layers and before the softmax. A start-up test asserts that changing future tokens never changes earlier outputs. Peak LR was chosen on validation from {1e-3, 3e-3} for the recurrent models (RNN 1e-3: val PPL 228 vs 240; LSTM 3e-3: 148 vs 201); the Transformer used 5e-4 (one value).

Cost measurement. Training time sums the training loops only (validation excluded, device synchronised). Inference uses one common workload for all models, re-timed back to back in one process: scoring the whole test set under the common setup (batch 32) and generating 50 tokens for each of 5 prompts (batch 1), one warm-up run, median of 3. The Transformer re-encodes its window at every generation step (no KV cache); RNN/LSTM update their state.

Prediction quality (Table 1). OBS LSTM (138.4) and Transformer (140.5) are tied—0.015 nats apart, which one seed cannot separate—and both are far better than the RNN (214.2), which is no better than counting 4-grams (216.7). The Transformer matches the LSTM with 28% fewer parameters; most parameters in every model sit in the vocabulary-sized embedding and softmax layers.

Cost. OBS The neural models need ~600–700× the n-gram's training time and are 10–15× slower at scoring despite the GPU. The Transformer is the fastest neural model at batched scoring (one parallel pass per chunk) but the slowest at token-by-token generation (62 vs 136–146 tok/s), because without a KV cache it recomputes the whole window at every step; this gap grows with output length.

Learning behaviour (Figure 1). OBS The Transformer converges fastest (it passes the 4-gram after 2 epochs) and has flattened by epoch 9 (val CE 5.018→5.017), while its training CE keeps falling: the final train−val gap is −0.36 nats *with dropout on*, i.e. it is starting to overfit. The LSTM starts much slower (val PPL 527 vs 313 after epoch 1), catches the Transformer at epoch 8 and is still improving at epoch 10 (−0.013 nats in the last epoch; gap −0.27). The RNN sits on a plateau near unigram quality for two epochs (val PPL ≈950, worse than the KN unigram's 818), escapes in epoch 3 together with a gradient-norm spike to 41 (clipped to 1), and still under-fits at epoch 10 (training CE above validation by 0.10). The LSTM is LR-sensitive: at 1e-3 it ends at test PPL 187, a larger effect than the LSTM–Transformer difference. HYP The RNN's plateau and spike are consistent with vanishing/exploding gradients through 128 tanh steps, which gating avoids. With a longer budget the LSTM might overtake the Transformer; with a shorter one the Transformer clearly wins.

Generated text (Table 2; same 5 prompts, same decoding: temperature 0.8, top-k 40, 50 tokens, <unk> banned, one RNG seed per prompt; more samples in Appendix B). OBS The 4-gram is locally fluent over 3–5 words but changes topic every few words (". . . andy murray in the final . federer won . . . a one-man show the film . . . the allied forces"); it never repeats a trigram. The RNN produces WikiText markup (`= = = and other = = =`), broken phrases and topic drift. The

Table 1: Part I: prediction quality and cost (test set, common chunked setup). Params: total (non-embedding, non-softmax body). Throughput in tokens/s; n-gram on CPU, neural models on the M1 GPU. Timing differences below ~10% (training) or ~25% (inference) are within the noise we measured on this laptop.

Model	Size	Test CE	**Test PPL**	Train time	Scoring tok/s	Generation tok/s
4-gram Kneser–Ney	5.8M n-grams (81 MB)	5.379	216.7	5 s	301,407	470
RNN	24.4M (0.94M)	5.367	214.2	56 min	19,953	146
LSTM	27.3M (3.77M)	**4.930**	**138.4**	59 min	25,472	136
Transformer	19.6M (2.21M)	4.945	140.5	53 min	29,345	62

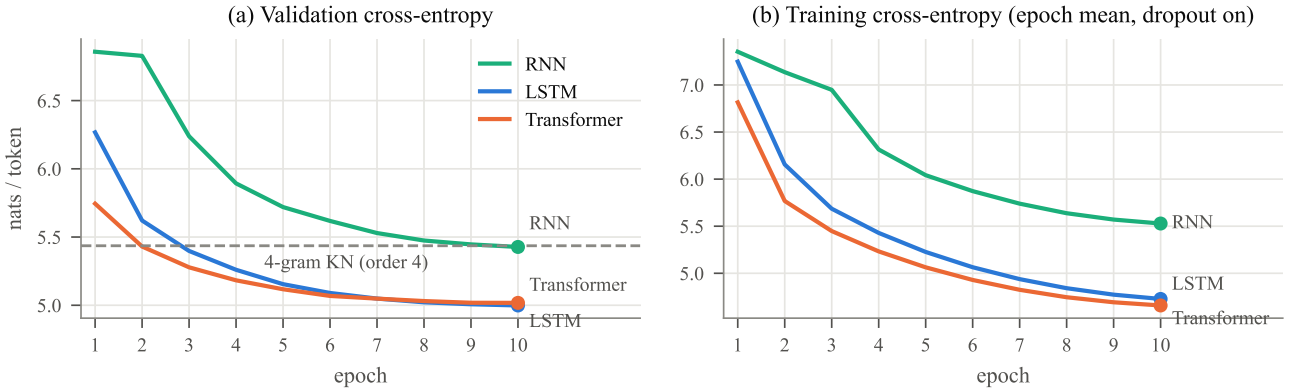


Figure 1: Part I learning curves. (a) Validation CE per epoch; dashed: 4-gram validation CE. (b) Training CE (epoch average, with dropout active).

Table 2: Part I: continuations of the prompt "*the album was released in*" (first 25 tokens; / = <eos>) and diversity over all 5 prompts (D-1/D-2: distinct uni-/bigram ratio; Rep: share of repeated trigrams). Same decoding and RNG seed for every model.

Model	Continuation	D-1	D-2	Rep
4-gram	late 1977 . the loss of the physical world of democratic union with the american society of composers . a study has become the city of nijmegen . . .	0.72	0.95	0.000
RNN	the 19th century , with a new cyclone in the country , with the new zealand was released by the northern terminus of the south of mexico . . .	0.64	0.91	0.025
LSTM	december 1940 , and was released in february 2003 , before the show was not released in october 2014 . at the end of the season . . .	0.65	0.93	0.004
Transformer	july 1992 , and was released in december 2005 , 2003 . a single was released on july 24 , 2014 . in the album was certified gold in japan . . .	0.56	0.86	0.037

LSTM and Transformer stay on topic: "the album was released in" → release dates, singles and certifications; "during the war , the army" → regiments, brigades and divisions. They differ sharply in repetition at equal PPL: the Transformer repeats content words ("the dam on the dam was built in the dam"; 3.7% of trigrams repeated) while the LSTM almost never does (0.4%); the LSTM's errors are semantic instead (release dates 63 years apart in one sentence). HYP Attention can copy any earlier token directly, which makes self-reinforcing copy loops easy; the LSTM must carry a word through its compressed state.

Trade-offs. The n-gram is the right choice when training and serving cost dominate or no GPU is available; it is only matched by the vanilla RNN. For quality, LSTM and Transformer tie on PPL; the Transformer reaches it with fewer parameters and far fewer updates and scores faster, while the LSTM generates faster (without a KV cache) and with less repetition.

An implementation pitfall (lesson learned). When re-evaluating saved checkpoints in a fresh process, the LSTM's test loss did not match the value logged during training (by −0.13 to +0.04 nats), while RNN and Transformer matched to 4 decimals. We traced this to PyTorch 2.5.1's MPS kernel for a multi-layer `nn.LSTM`: its built-in inter-layer dropout was not resampled between calls, and evaluation at the training tensor shape reused the train-mode graph. We rebuilt the LSTM from one single-layer `nn.LSTM` per layer with an explicit dropout module (same parameters), added a start-up check (dropout must resample in training; evaluation must match a CPU copy), retrained every LSTM, and confirmed that a fresh CPU evaluation reproduces each logged test loss. With the bug the LSTM appeared much worse (test PPL 172 at LR 3e-3), which would have produced a wrong "Transformer ≫ LSTM" conclusion.

3 Part II: input-embedding variants (LSTM)

Design. We keep the Part I LSTM and change only its input-embedding table (dimension 300, chosen so that GloVe-300d fits without projection): **A** random and trainable (the Part I run, reused); **B** our own Skip-gram [6] vectors trained on the LM training split (window 5, 5 negatives, 10 epochs; gensim [11]) and **frozen**; **C** public GloVe 6B-300d [7] (Wikipedia + Gigaword, ~6B tokens, ≈2,900× our corpus), **frozen**. Freezing covers the whole table (`requires_grad=False`, excluded

Table 3: Part II: one LSTM, three input-embedding settings (all best at epoch 10; no non-finite gradients in any run).

Setting	Trainable	Test CE	**Test PPL**	Val PPL ep. 1 / 2 / 3	Train CE	Train−val	Max grad norm
A random, trainable	27.3M	4.930	138.4	527 / 276 / 221	4.73	−0.27	2.18
B Skip-gram, frozen	18.6M	**4.688**	**108.6**	452 / 195 / 151	4.27	−0.47	1.95
C GloVe, frozen	18.6M	4.701	110.1	790 / 234 / 166	4.35	−0.41	1.78

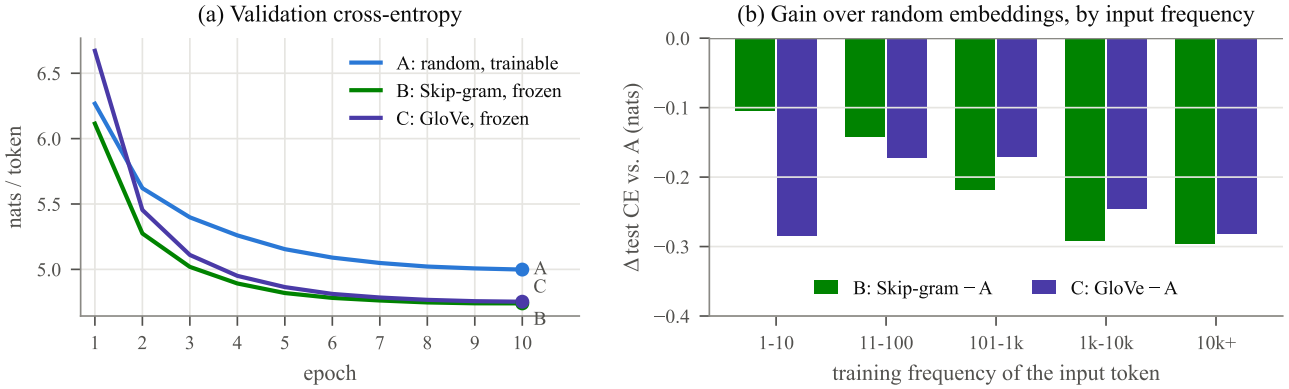


Figure 2: Part II. (a) Validation CE per epoch. (b) Change in test CE relative to A, grouped by how often the *input* token (the one before the target) occurs in training (bucket shares of test targets: 3%, 14%, 24%, 15%, 36%).

from the optimiser); trainable parameters drop from 27.3M to 18.6M. Data, architecture, LR (3e-3, selected for A), schedule and seed are identical. Each frozen table is multiplied by one scalar so that its mean row norm equals that of A's learned table (2.87); this keeps cosine geometry and puts all settings on the same input scale. *Coverage*: Skip-gram covers 100% of the vocabulary. GloVe covers 96.2% of word types and 95.8% / 92.5% of train / test tokens after mapping WikiText's `@-@`, `@,@`, `@.@` to `-`, `,`, `.`; the uncovered tokens are mostly <unk> and <eos>, plus 1,098 rare types (*'t*, °, rare names: 0.1% of test inputs). Missing rows are drawn from $\mathcal{N}(\mu_d, \sigma_d)$ of the GloVe vectors (seed 42) and frozen too. *Cost*: training the Skip-gram vectors took 27–38 s on the CPU (≈1% of one LM run); GloVe needs a 394 MB download and no training on our side (its own pre-training cost is far larger).

Results (Table 3, Figure 2). OBS Both frozen pre-trained tables beat the jointly learned one by a wide margin (−0.24 / −0.23 nats, 20–22% lower PPL); B and C are within 0.013 nats. *Convergence*: B leads from the first epoch; C starts slowest (val PPL 790 after epoch 1) but overtakes A during epoch 2 and approaches B every epoch. *Stability*: no setting shows spikes or non-finite gradients; maximum gradient norms are 1.8–2.2, so stability does not separate them here. *Fit*: the frozen models reach a much *lower* training CE with fewer trainable parameters and also generalise better, with a *larger* train−val gap—freezing did not act mainly as a regulariser but made the LM easier to optimise. *Where the gain comes from* (Fig. 2b): the frozen tables help in every frequency band, most in absolute terms for frequent inputs (−0.30 nats for words seen >10k times). GloVe is clearly best for the rarest inputs (words seen ≤10 times: −0.29 vs −0.10 for Skip-gram), Skip-gram for frequent and mid-frequency ones. *Representations*: A's learned table groups closed classes well (months, numbers) but is noisy for many content words (nearest neighbours of *war*: *fastra, ahk, mahdi*); Skip-gram gives corpus-specific topical neighbours (*river* → *missouri, mississippi*); GloVe gives clean general semantic neighbours (*war* → *wars, conflict, battle*) (Appendix C). *Generation*: the frozen models stay closer to the prompt ("scientists have found that the" → species, kakapo, allosaurus; A drifts to a film soundtrack); C repeats most (3.8% repeated trigrams vs 1.3% B, 0.4% A).

Interpretation: trainability vs. extra data. *A vs B* (same 2.08M tokens, trainable vs pre-trained-and-frozen): under our 10-epoch budget, end-to-end trainability did not pay off; a 30-second Skip-gram pass plus freezing was worth 0.24 nats. HYP Each embedding row only receives gradient when its word occurs, so most of the 28.9k rows get few noisy updates in 5k steps, whereas Skip-gram is built to learn exactly these vectors from co-occurrence. *B vs C* (both frozen; ≈2,900× more general-domain data, but 7.5% of test tokens without a real vector): a tie overall; the extra data helps exactly where it should—rare words—while in-domain Skip-gram wins on frequent words. More data is not automatically better when the small corpus *is* the target domain. *Not separable here*: whether the gain comes from the pre-trained *initialisation* or from *freezing*; a fourth run with Skip-gram initialisation and trainable embeddings would answer this.

4 Part III: downstream topic classification

Task and protocol. AG News [8] (4 balanced news topics). From the official training set we draw a balanced 20,000-document training set and a disjoint 5,000-document validation set (seed 42); the official 7,600-document test set is used once. Text is preprocessed like WikiText (HTML entities removed, lower-cased, `1 @,@ 000`, `long @-@ term`, `wasn 't`, punctuation split, <eos> appended); 6.8% of tokens fall outside the LM vocabulary and become <unk>; documents average 45 tokens (17 test documents exceed the 128-token context and are truncated). All five LMs stay frozen. **Fixed-length representations**: *mean* = average of the top-layer token states over the document's real tokens (padding excluded); *last* = the top-layer state at the final <eos>, the only position of these causal models that has read the whole document. Baselines: the average of input-embedding rows (no context), and TF-IDF uni+bigrams over the original words (no

LM, no <unk>). **Classifier** (identical for all): standardised features + multinomial logistic regression (scikit-learn [12]), $C \in \{0.01, 0.1, 1, 10\}$ chosen on validation accuracy; metrics: test accuracy and macro-F1 (they agree within 0.006 because classes are balanced).

Table 4: Part III: AG News test results with 20k training documents (chance 0.25). Validation and test accuracy agree within 1.2 points for every row. "Emb. avg": mean of the input-embedding rows (no context).

Representation	LM test PPL	Mean pooling		Last token	
		Acc.	Macro-F1	Acc.	Macro-F1
RNN	214.2	0.661	0.659	0.399	0.394
LSTM (random emb.)	138.4	0.719	0.718	0.617	0.615
Transformer	140.5	0.780	0.779	0.769	0.768
LSTM + Skip-gram (frozen)	108.6	0.809	0.808	0.785	0.785
LSTM + GloVe (frozen)	110.1	**0.848**	**0.847**	**0.820**	**0.820**
Emb. avg: LSTM's learned table / Skip-gram / GloVe	–	0.761 / 0.828 / 0.878		(macro-F1 0.760 / 0.827 / 0.878)	
TF-IDF uni+bigrams (no LM)	–	**0.900**		(macro-F1 0.900)	

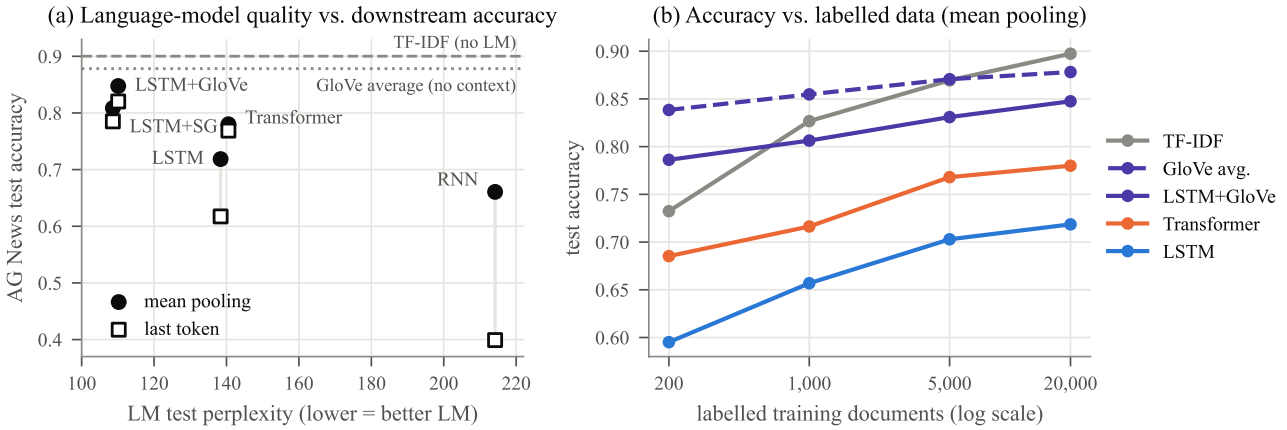


Figure 3: Part III. (a) LM test perplexity vs. AG News test accuracy for both poolings (vertical segments join the two poolings of one LM). (b) Test accuracy vs. number of labelled training documents (balanced prefixes of the training set; C re-selected on the same validation set).

Does the LM ranking agree with the downstream ranking? Only partly (Table 4, Fig. 3a). OBS Spearman's ρ between LM test PPL and accuracy is -0.80 for both poolings (n=5, p=0.10): the worst LM gives the worst features and the two frozen-embedding LSTMs the best. But where perplexities tie, downstream accuracy does not: the Transformer beats the LSTM by 6.1 points (mean) and 15.1 (last), and LSTM+GloVe beats LSTM+Skip-gram by 3.9 points although its PPL is slightly worse. The downstream order follows the embedding source and the architecture more than PPL.

Pooling and context. OBS Mean pooling beats last-token pooling for every LM, by 26.2 points for the RNN (whose final state is near chance: 39.9%), 10.1 for the LSTM, 2.4–2.7 for the frozen-embedding LSTMs and 1.1 for the Transformer, whose last position attends to every token directly. Strikingly, for every LSTM the *context-free* average of its own input embeddings beats its mean-pooled hidden states (GloVe 87.8 vs 84.8; Skip-gram 82.8 vs 80.9; learned 76.1 vs 71.9), although the hidden states are larger (512-d vs 300-d); the Transformer's states (78.0) do beat the LSTM's learned-table average. HYP Recurrent layers trained only for next-word prediction re-encode their input around local, predictive information and lose some linearly accessible topic information; the Transformer's residual stream carries each token's embedding to the top layer and keeps more lexical identity. GloVe's advantage downstream (but not in PPL) fits its training on general newswire-like text, whose geometry encodes topical similarity.

Amount of labelled data (Fig. 3b). OBS With 200 labelled documents, GloVe averages (83.8%) and GloVe-LSTM states (78.6%) beat TF-IDF (73.2%) by a wide margin; TF-IDF catches up at about 5,000 documents and wins at 20,000 (89.7–90.0%). Pre-trained dense representations matter most when labels are scarce.

Errors. OBS Business↔Sci/Tech is the dominant confusion for every representation (515 of the 1,159 errors of LSTM+GloVe/mean; 338 of 759 for TF-IDF), e.g. "AOL to cut 700 jobs" (Business, predicted Sci/Tech) or "Dell may soon unveil more consumer goods" (Sci/Tech, predicted Business; Table 5); some labels are debatable (a Sony/IBM chip announcement labelled Business; a cricket test match labelled World), and 224 test documents are misclassified by every LM representation. TF-IDF is right where the best LM representation is wrong for 621 documents (221 the other way); these typically hinge on one decisive word or source tag (*ESPN*, *NBA*, *VoIP*, *AFP*—the last is <unk> for the LM), which a 45-token average dilutes. The LM representation wins on documents without a keyword but with clear topical vocabulary ("Canada defends submarine fleet", "pass interference on a pivotal interception"). The share of <unk> tokens barely matters

(83.4–85.2% accuracy from 0% to >10% <unk>).

Table 5: Part III: representative test errors of the best LM representation (LSTM+GloVe, mean pooling) and of TF-IDF.

True	LSTM+GloVe / TF-IDF	Text (truncated)
Sci/Tech	Business / **Sci/Tech**	Dell May Soon Unveil More Consumer Goods (Reuters) . . . the world's largest PC maker, could announce an expanded selection of its consumer electronics line . . .
Sports	Sci/Tech / **Sports**	ESPN.com news services: Six months ago, Scottie Pippen issued a "This is probably it for me" declaration . . . in an NBA uniform.
World	**World** / Sports	Canada defends submarine fleet: Canada has defended its decision to buy second-hand submarines after a crewman died . . .
Sports	**Sports** / World	SEC's Gaston Admits Official Missed Call (AP) . . . an LSU player should have been called for pass interference on a pivotal interception . . .
Business	Sci/Tech / Sci/Tech	Sony, IBM, Toshiba Give Details of 'Cell' Chip (Reuters) . . . (missed by every representation; debatable label)
World	Sports / Sports	Warne takes six but India establish handy lead (Reuters) . . . (missed by every representation; debatable label)

5 Discussion, limitations and lessons

Across parts. Perplexity is a good summary of next-token prediction but a coarse proxy for other qualities: two LM pairs with tied PPL differ clearly in sample repetition (Part I: LSTM vs Transformer) and in downstream accuracy (Part III: by 4–6 points). The single most effective change in this study was not the architecture but the input embeddings: a cheap in-domain Skip-gram table, frozen, improved PPL by 22% (Part II), and the representation that transferred best downstream was the one built on GloVe, whose extra data shows up on rare words (Part II) and on news topics (Part III).

Limitations.

- One seed per configuration: differences below ∼0.02 nats or ∼1 accuracy point are not interpretable (e.g. LSTM vs Transformer PPL, Skip-gram vs GloVe PPL, RNN vs 4-gram).
- Unequal tuning: two LRs for the recurrent models, one for the Transformer, and dropout 0.3 vs 0.2; hence "LSTM ≈ Transformer" rather than a ranking. The Part II LR was tuned for setting A only.
- A short 10-epoch budget favours fast-converging models; the chunked evaluation (no state across 128-token chunks) makes our PPLs incomparable with published WikiText-2 numbers.
- Part III uses frozen LMs and a linear classifier only; its 5-point correlation conflates architecture and embedding source; the LM vocabulary turns 6.8% of AG News tokens into <unk>, which favours TF-IDF.
- Laptop timings drift with temperature, sleep and power source.

Lessons.

- Cross-check logged metrics against an independent re-evaluation: it exposed a framework bug that had inverted a conclusion.
- Report the training recipe together with the architecture: a factor-3 LR change moved the LSTM by 48 PPL.
- For causal LMs, prefer mean pooling, and always check simple baselines: an average of good word vectors beat every contextual LM representation in this setting.

References

[1] S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer sentinel mixture models. *arXiv:1609.07843*, 2016.

[2] R. Kneser, H. Ney. Improved backing-off for m-gram language modeling. *ICASSP*, 1995.

[3] S. F. Chen, J. Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language* 13(4), 1999.

[4] S. Hochreiter, J. Schmidhuber. Long short-term memory. *Neural Computation* 9(8), 1997.

[5] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

[6] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. Distributed representations of words and phrases and their compositionality. *NeurIPS*, 2013.

[7] J. Pennington, R. Socher, C. D. Manning. GloVe: Global vectors for word representation. *EMNLP*, 2014.

[8] X. Zhang, J. Zhao, Y. LeCun. Character-level convolutional networks for text classification. *NeurIPS*, 2015.

[9] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

[10] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.

[11] R. Řehůřek, P. Sojka. Software framework for topic modelling with large corpora. *LREC Workshop on New Challenges for NLP Frameworks*, 2010.

[12] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR* 12, 2011.

A Configuration and reproducibility

	RNN / LSTM	Transformer
Input embedding	300-d, U(−0.1, 0.1), trainable (Part II: frozen B/C)	same
Body	2 layers, hidden 512 (LSTM: one `nn.LSTM` per layer)	4 pre-LN blocks, 6 heads, FFN 1200, GELU, learned positions
Dropout	0.3 (embedding, between layers, before softmax)	0.2
Output	untied linear, 28,913 classes	untied linear
Peak LR	RNN 1e-3, LSTM 3e-3 (validation-selected)	5e-4
Shared	context 128, batch 32, 10 epochs, AdamW [9] wd 0.01, 200 warm-up steps, cosine to 10%, clip 1.0, seed 42	

Commands, configurations, random seeds and data-access instructions are in the repository's `README.md` (PyTorch [10], gensim, scikit-learn). Each run stores `results.json` (metrics, parameter counts, timings, hardware, generations) and `history.json` (per-epoch losses, gradient norms); the per-part write-ups `PART1/2/3_RESULTS.md` contain the complete tables (e.g. per-class F1, learning-curve values, all generations) behind the numbers above. **Data and tools.** WikiText-2 (CC BY-SA) and AG News (academic use) are downloaded by the scripts and not redistributed; GloVe is downloaded through gensim-data. **Use of AI tools.** The code, experiment scripts, figures and a first draft of this report were produced with substantial assistance from Claude (Anthropic), an AI coding assistant.

B Generated text

Prompt "*during the war , the army*" (decoding as in Part I; / = <eos>):

LSTM: decided to leave the war . the german and ottoman regiment and the men had been reduced to the french and 7th light horse brigade . the ship was held on 15 february 1903 . the 3rd infantry division was given to a position . . .

Transformer: corps , which he was sent to a new army troops . the battle of the 10th battalion was appointed in mostar , but the 3rd battalion to attack the 1st infantry rifles regiments division of nevesinje brigade . . .

Prompt "*scientists have found that the*" (Part II settings):

A random: use of the " man " was not seen . the first of the film 's soundtrack , " you are the illinoise ! " . . .

B Skip-gram: presence of the species is not used to refer the word . / the noisy miner miner was also described as the dubious in the american red cross . . .

C GloVe: kakapo had " the same time to have been the same of the more expensive " than the same species . / corythosaurus muscaria is a very common species of allosaurus . . .

Diversity over all 5 prompts for the Part II settings (distinct-1 / distinct-2 / repeated-trigram rate): A (= Part I LSTM) 0.65 / 0.93 / 0.004; B 0.68 / 0.94 / 0.013; C 0.65 / 0.89 / 0.038.

C Nearest neighbours in the input-embedding tables (cosine)

Word (train freq.)	A: learned with the LM	B: Skip-gram	C: GloVe
war (1,370)	fastra, ahk, mahdi, bishopric	civil, outbreak, airbase, conflict	wars, conflict, battle, civil
guitar (400)	dj, asylums, saiban, pinyin	hero, drums, metallica, aerosmith	guitars, bass, vocals, piano
river (861)	fort, grape, creek, bundle	missouri, mississippi, platte, susquehanna	rivers, tributary, creek, valley
january (693)	november, october, february, june	november, march, february, april	february, december, october, november
three (1,976)	two, four, six, five	seven, eight, five, thirteen	four, five, two, six
scientists (57)	operatives, scholars, they, divines	exoplanets, nasa, resuscitation	researchers, experts, biologists